Maze Solver with Hazard Environment

COSC 4368 — AI Project Check-in 2

Group 15 - Marlon Melara, Ahnaf Murshid, Israel Mummay, Brittany Picazo, Osinachi Enemuoh

March 25, 2026

Project Goal

- Reconstruct maze from provided image
- Convert maze to a logical **64x64 grid**
- Solve the maze without hazards
- Load hazards into the environment
- Demonstrate hazard behavior

Checkpoint focus: environment setup, solver, visualization, and hazard system.

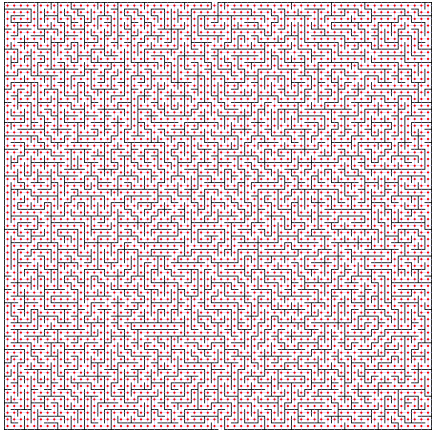
Maze Reconstruction

Pipeline

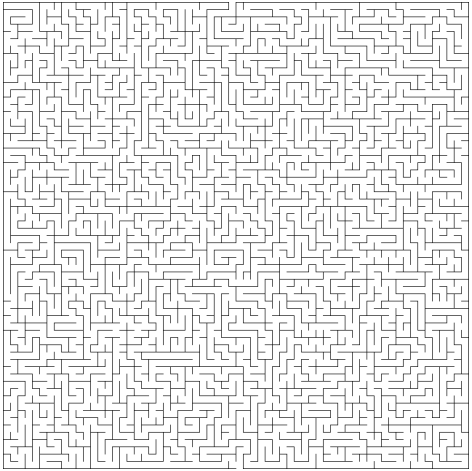
- Extract maze from image
- Detect wall boundaries
- Convert into logical grid
- Build adjacency structure

Generated files:

- `maze_cells.npy`
- `maze_walls_right.npy`
- `maze_walls_down.npy`



64x64 Logical Maze



This visualization confirms the extracted logical maze structure.

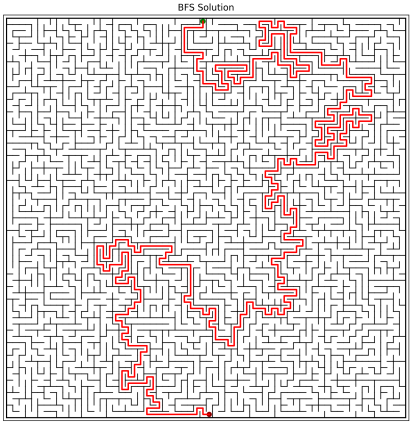
Naive Solver (BFS)

Breadth First Search was used because:

- guarantees shortest path
- simple and deterministic
- ideal for grid mazes

Results:

- Path length: **423 cells**
- Moves required: **422**
- Turns (max 5 actions): **85**



Hazard System

Three hazard types:

Death Pits

- instant death
- respawn at start

Teleport Pads

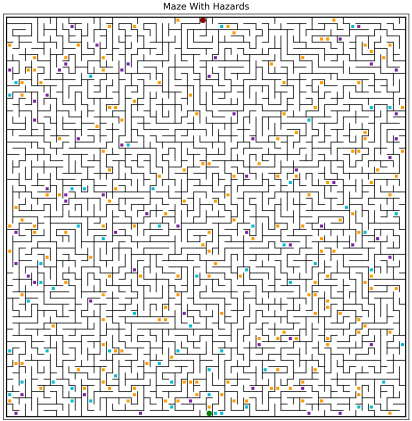
- move agent to mapped cell

Confusion Pads

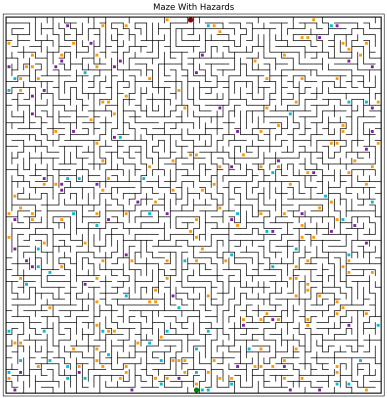
- invert controls temporarily

Hazard density:

$\approx 7\%$ of navigable cells



Hazard Legend



Element	Marker	Meaning
●	Green circle	Start
●	Red circle	Goal
—	Red line	BFS path
■	Orange square	Death pit
■	Purple square	Teleport pad
■	Cyan square	Confusion pad

Demo Results

- Maze successfully reconstructed
- BFS solver found valid path
- Hazard density within required range
- Death pit demo triggered correctly
- Teleport demo triggered correctly
- Confusion demo triggered correctly

The system now behaves as a grid-world AI environment.

Next Steps

Future work:

- Implement intelligent navigation agents
- Hazard-aware path planning
- Reinforcement learning exploration

Questions?